In [1]:
```
# CELL 0
# OBTAINING AND LOADING DATA
import pandas as pd

# Set display options for better readability
pd.options.display.float_format = "{:.2f}".format

# Load the datasets
marketing_ads_data = pd.read_csv("marketing_ads_data.csv")
sales_transaction_data = pd.read_csv("sales_transaction_data.csv")

print("Data Loads Successfully")
```

```
Data Loads Successfully
```

In [2]:
```
# CELL 1
# SCRUBBING AND PREPARING DATA

print("===== MARKETING ADS DATA =====")

# Data Structure
marketing_ads_data.info()

# Basic Statistics
print(marketing_ads_data.describe())

# Missing Values Check
print(marketing_ads_data.isnull().sum())

# DUplicate Check
print("Duplikat:", marketing_ads_data.duplicated().sum())

# Unique Values Check for Important Columns
print("Variant:", marketing_ads_data['variant'].unique())
print("Campaign Name:", marketing_ads_data['campaign_name'].unique())
print("Audience Type:", marketing_ads_data['audience_type'].unique())
print("Product Type:", marketing_ads_data['product_type'].unique())
print("Product Name:", marketing_ads_data['product_name'].unique())

# Data Sample
print(marketing_ads_data.sample(5))


print("===== SALES TRANSACTION DATA =====")

# Data Structure
sales_transaction_data.info()

# Basic Statistics
print(sales_transaction_data.describe())

# Missing Values Check
print(sales_transaction_data.isnull().sum())

# DUplicate Check
```

```
print("Duplikat:", sales_transaction_data.duplicated().sum())

# Unique Values Check for Important Columns
print("Variant:", sales_transaction_data['variant'].unique())
print("Product Type:", sales_transaction_data['product_type'].unique())
print("Product Name:", sales_transaction_data['product_name'].unique())
print("Order Status:", sales_transaction_data['order_status'].unique())

# Data Sample
print(sales_transaction_data.sample(5))
```

```
===== MARKETING ADS DATA =====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1030 entries, 0 to 1029
Data columns (total 12 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   ad_id          1030 non-null   object
 1   campaign_name  1030 non-null   object
 2   audience_type  1030 non-null   object
 3   product_type   1030 non-null   object
 4   product_name   1030 non-null   object
 5   variant        1030 non-null   object
 6   date           1030 non-null   object
 7   impressions    1030 non-null   int64
 8   clicks         1030 non-null   int64
 9   spend_idr      970 non-null    float64
 10  sessions       1030 non-null   int64
 11  add_to_cart    1030 non-null   int64
dtypes: float64(1), int64(4), object(7)
memory usage: 96.7+ KB
       impressions  clicks  spend_idr  sessions  add_to_cart
count      1030.00 1030.00     970.00   1030.00      1030.00
mean      12302.35  228.75  309504.70    194.00        23.62
std        3796.80  122.20  193472.87    104.47        16.45
min        5078.00   47.00    5048.00     38.00         3.00
25%        9206.25  135.00  197517.50    115.00        12.00
50%       12013.00  200.50  300263.50    171.00        19.00
75%       15052.25  296.50  404518.50    250.75        31.00
max       19940.00  624.00 2839487.00    536.00        99.00
ad_id             0
campaign_name     0
audience_type     0
product_type      0
product_name      0
variant           0
date              0
impressions       0
clicks            0
spend_idr        60
sessions          0
add_to_cart       0
dtype: int64
Duplikat: 30
Variant: ['Video_Review' 'Static_Catalog']
Campaign Name: ['Awareness Campaign' 'Retargeting Campaign' 'Flash Sale Campaign']
Audience Type: ['cold' 'warm']
Product Type: ['Serum' 'Sunscreen' 'Cleanser' 'Moisturizer']
Product Name: ['DermaGlow Brightening Serum' 'DermaGlow UV Shield Sunscreen'
 'DermaGlow Gentle Cleanser' 'DermaGlow Hydrating Moisturizer']
       ad_id         campaign_name audience_type product_type  \
315  AD_0240  Retargeting Campaign          warm  Moisturizer
916  AD_0493   Flash Sale Campaign          cold  Moisturizer
783  AD_0923   Flash Sale Campaign          cold    Sunscreen
471  AD_0368  Retargeting Campaign          warm     Cleanser
560  AD_0154    Awareness Campaign          cold        Serum
```

```
                        product_name         variant          date  \
315  DermaGlow Hydrating Moisturizer  Static_Catalog    2023-05-20
916  DermaGlow Hydrating Moisturizer  Static_Catalog    2023-01-05
783    DermaGlow UV Shield Sunscreen  Static_Catalog    2023-06-09
471        DermaGlow Gentle Cleanser    Video_Review    15/03/2023
560      DermaGlow Brightening Serum    Video_Review  Jan 31, 2023

     impressions  clicks  spend_idr  sessions  add_to_cart
315         5769      73  208212.00        65            8
916         8716     102  343767.00        82           11
783         7780      92  232475.00        73           10
471        15155     475  267849.00       418           23
560        14561     433  142534.00       353           60
===== SALES TRANSACTION DATA =====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   transaction_id    2500 non-null   object
 1   ad_id             2408 non-null   object
 2   transaction_date  2500 non-null   object
 3   product_type      2500 non-null   object
 4   product_name      2500 non-null   object
 5   variant           2500 non-null   object
 6   revenue_idr       2500 non-null   float64
 7   order_status      2500 non-null   object
dtypes: float64(1), object(7)
memory usage: 156.4+ KB
       revenue_idr
count      2500.00
mean     927305.40
std    16501251.90
min     -445000.00
25%       86750.00
50%      217500.00
75%      358000.00
max   469235332.00
transaction_id       0
ad_id               92
transaction_date     0
product_type         0
product_name         0
variant              0
revenue_idr          0
order_status         0
dtype: int64
Duplikat: 0
Variant: ['Static_Catalog' 'Video_Review']
Product Type: ['Cleanser' 'Moisturizer' 'Serum' 'Sunscreen']
Product Name: ['DermaGlow Gentle Cleanser' 'DermaGlow Hydrating Moisturizer'
 'DermaGlow Brightening Serum' 'DermaGlow UV Shield Sunscreen']
Order Status: ['Success' 'Cancelled' 'Refunding']
     transaction_id    ad_id transaction_date product_type  \
1446     TXN_001449  AD_0274       2023-03-16     Cleanser
729      TXN_001878  AD_0471       2023-07-26  Moisturizer
```

```
904      TXN_001935  AD_0942       2023-08-11     Cleanser
1510     TXN_002168  AD_0367       2023-12-19        Serum
533      TXN_001903  AD_0084       2023-12-31    Sunscreen

                         product_name         variant  revenue_idr  \
1446        DermaGlow Gentle Cleanser  Static_Catalog    358000.00
729   DermaGlow Hydrating Moisturizer  Static_Catalog    407000.00
904         DermaGlow Gentle Cleanser  Static_Catalog    446000.00
1510      DermaGlow Brightening Serum  Static_Catalog    360000.00
533     DermaGlow UV Shield Sunscreen  Static_Catalog     94000.00

     order_status
1446      Success
729       Success
904       Success
1510      Success
533       Success
```

In [3]:

```python
# CELL 2
# SCRUBBING 2 - Filtering.
# Convert date columns to datetime format
marketing_ads_data['date'] = pd.to_datetime(
    marketing_ads_data['date'],
    format='mixed',
    dayfirst=True
)

sales_transaction_data['transaction_date'] = pd.to_datetime(
    sales_transaction_data['transaction_date'],
    format='mixed',
    dayfirst=True
)

# Handle Duplicate Rows
marketing_ads_data = marketing_ads_data.drop_duplicates()

# FIll missing values in spend_idr column with Median
marketing_ads_data['spend_idr'] = marketing_ads_data.groupby('campaign_name')['spen
    lambda x: x.fillna(x.median())
)

print("Hasil data cleaning pada marketing_ads_data:")
print(marketing_ads_data.info())
print("Shape setelah drop duplicate:", marketing_ads_data.shape)
print("Hasil data cleaning pada sales_transaction_data:")
print(sales_transaction_data.info())
print("\n Jumlah Null di Spend_idr setelah diisi dengan median:", marketing_ads_dat
```

```
Hasil data cleaning pada marketing_ads_data:
<class 'pandas.core.frame.DataFrame'>
Index: 1000 entries, 0 to 1029
Data columns (total 12 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   ad_id          1000 non-null   object
 1   campaign_name  1000 non-null   object
 2   audience_type  1000 non-null   object
 3   product_type   1000 non-null   object
 4   product_name   1000 non-null   object
 5   variant        1000 non-null   object
 6   date           1000 non-null   datetime64[ns]
 7   impressions    1000 non-null   int64
 8   clicks         1000 non-null   int64
 9   spend_idr      1000 non-null   float64
 10  sessions       1000 non-null   int64
 11  add_to_cart    1000 non-null   int64
dtypes: datetime64[ns](1), float64(1), int64(4), object(6)
memory usage: 101.6+ KB
None
Shape setelah drop duplicate: (1000, 12)
Hasil data cleaning pada sales_transaction_data:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   transaction_id    2500 non-null   object
 1   ad_id             2408 non-null   object
 2   transaction_date  2500 non-null   datetime64[ns]
 3   product_type      2500 non-null   object
 4   product_name      2500 non-null   object
 5   variant           2500 non-null   object
 6   revenue_idr       2500 non-null   float64
 7   order_status      2500 non-null   object
dtypes: datetime64[ns](1), float64(1), object(6)
memory usage: 156.4+ KB
None

 Jumlah Null di Spend_idr setelah diisi dengan median: 0
```

In [4]:
```
# CELL 3
# SCRUBBING (Mixed date formats) *Before conversion, check unique values in the 'da
marketing_ads_data['date'].unique()
```

Out[4]:
```
<DatetimeArray>
['2023-01-06 00:00:00', '2023-09-01 00:00:00', '2023-04-13 00:00:00',
 '2023-11-14 00:00:00', '2023-04-15 00:00:00', '2023-03-29 00:00:00',
 '2023-03-16 00:00:00', '2023-12-09 00:00:00', '2023-08-08 00:00:00',
 '2023-12-23 00:00:00',
 ...
 '2023-01-02 00:00:00', '2023-02-21 00:00:00', '2023-12-19 00:00:00',
 '2023-08-06 00:00:00', '2023-05-01 00:00:00', '2023-05-15 00:00:00',
 '2023-02-10 00:00:00', '2023-08-19 00:00:00', '2023-09-18 00:00:00',
 '2023-10-20 00:00:00']
Length: 344, dtype: datetime64[ns]
```

In [5]:
```
# CELL 4
# [SCRUBBING] Checking missing values in ad_id column of sales_transaction_data to

missing_ad_id = sales_transaction_data[sales_transaction_data['ad_id'].isnull()]
print("Jumlah Missing:", len(missing_ad_id))
print(missing_ad_id.head(10))
```

```
Jumlah Missing: 92
    transaction_id ad_id transaction_date product_type  \
0       TXN_001448   NaN       2023-03-15     Cleanser
8       TXN_000498   NaN       2023-05-26     Cleanser
37      TXN_001367   NaN       2023-05-20    Sunscreen
48      TXN_001856   NaN       2023-02-10  Moisturizer
57      TXN_001294   NaN       2023-09-14     Cleanser
71      TXN_002347   NaN       2023-07-17    Sunscreen
98      TXN_001018   NaN       2023-09-15    Sunscreen
113     TXN_002115   NaN       2023-12-04    Sunscreen
151     TXN_001468   NaN       2023-11-28        Serum
155     TXN_001969   NaN       2023-08-12     Cleanser

                        product_name         variant  revenue_idr order_status
0          DermaGlow Gentle Cleanser  Static_Catalog    403000.00      Success
8          DermaGlow Gentle Cleanser    Video_Review         0.00    Cancelled
37     DermaGlow UV Shield Sunscreen    Video_Review    394000.00      Success
48   DermaGlow Hydrating Moisturizer  Static_Catalog   -147000.00    Refunding
57         DermaGlow Gentle Cleanser    Video_Review    123000.00      Success
71     DermaGlow UV Shield Sunscreen    Video_Review    394000.00      Success
98     DermaGlow UV Shield Sunscreen    Video_Review    186000.00      Success
113    DermaGlow UV Shield Sunscreen  Static_Catalog     86000.00      Success
151      DermaGlow Brightening Serum  Static_Catalog    313000.00      Success
155        DermaGlow Gentle Cleanser  Static_Catalog         0.00    Cancelled
```

In [6]:
```
# CELL 5
# [SCRUBBING] Checking unique values in order_status column to identify any inconsi

print(sales_transaction_data['order_status'].value_counts())
```

```
order_status
Success      1936
Cancelled     355
Refunding     209
Name: count, dtype: int64
```

In [7]:
```
# CELL 6
# [SCRUBBING] Filter out rows where order_status is not 'Success'
```

```
sales_transaction_clean = sales_transaction_data[sales_transaction_data['order_stat
print("Data sebelum filter:", len(sales_transaction_data))
print("Data setelah filter:", len(sales_transaction_clean))
print("Transaksi ekslusif:", len(sales_transaction_data) - len(sales_transaction_cl
```

```
Data sebelum filter: 2500
Data setelah filter: 1936
Transaksi ekslusif: 564
```

In [8]:
```
# CELL 7
# [SCRUBBING] Fill Missing Values in ad_id with 'Organic' (BEFORE)

print("missing_ad_id setelah filter:", sales_transaction_clean['ad_id'].isnull().su
```

```
missing_ad_id setelah filter: 70
```

In [9]:
```
# CELL 8
# [SCRUBBING] Fill Missing Values in ad_id with 'Organic' (AFTER)

sales_transaction_clean = sales_transaction_clean.copy()
sales_transaction_clean['ad_id'] = sales_transaction_clean['ad_id'].fillna('Organic

print("Missing_ad_id setelah diisi:", sales_transaction_clean['ad_id'].isnull().sum
```

```
Missing_ad_id setelah diisi: 0
```

In [10]:
```
# CELL 9
# [SCRUBBING] Revenue Check after filter

print(sales_transaction_clean['revenue_idr'].describe())
print("Nilai Negatif:", (sales_transaction_clean['revenue_idr'] < 0).sum())
```

```
count        1936.00
mean      1218387.66
std      18742396.20
min         65000.00
25%        173000.00
50%        286000.00
75%        389000.00
max     469235332.00
Name: revenue_idr, dtype: float64
Nilai Negatif: 0
```

In [11]:
```
# CELL 10
# [SCRUBBING]

sales_transaction_clean = sales_transaction_clean[sales_transaction_clean['revenue_
print("Shape setelah drop outlier:", sales_transaction_clean.shape)
```

```
Shape setelah drop outlier: (1931, 8)
```

In [12]:
```
# CELL 11
# [SCRUBBING] FINAL SANITY CHECKS - Marketing Ads Data & Sales Transaction Data

print("==== FINAL SANITY CHECKS MARKETING ADS DATA =====")
print("Shape:", marketing_ads_data.shape)
print("Missing Values:\n", marketing_ads_data.isnull().sum())
```

```
print("Dtype:\n", marketing_ads_data.dtypes)
print("Cek Duplikat:", marketing_ads_data.duplicated().sum())

print("==== FINAL SANITY CHECKS SALES TRANSACTION DATA =====")
print("Shape:", sales_transaction_clean.shape)
print("Missing Values:\n", sales_transaction_clean.isnull().sum())
print("Dtype:\n", sales_transaction_clean.dtypes)
print("Order_status:", sales_transaction_clean['order_status'].unique())
print("Ad_id:", sales_transaction_clean["ad_id"].isnull().sum())
```

```
==== FINAL SANITY CHECKS MARKETING ADS DATA =====
Shape: (1000, 12)
Missing Values:
 ad_id            0
campaign_name    0
audience_type    0
product_type     0
product_name     0
variant          0
date             0
impressions      0
clicks           0
spend_idr        0
sessions         0
add_to_cart      0
dtype: int64
Dtype:
 ad_id                   object
campaign_name            object
audience_type            object
product_type             object
product_name             object
variant                  object
date             datetime64[ns]
impressions               int64
clicks                    int64
spend_idr               float64
sessions                  int64
add_to_cart               int64
dtype: object
Cek Duplikat: 0
==== FINAL SANITY CHECKS SALES TRANSACTION DATA =====
Shape: (1931, 8)
Missing Values:
 transaction_id      0
ad_id               0
transaction_date    0
product_type        0
product_name        0
variant             0
revenue_idr         0
order_status        0
dtype: int64
Dtype:
 transaction_id              object
ad_id                       object
transaction_date    datetime64[ns]
product_type                object
product_name                object
variant                     object
revenue_idr                float64
order_status                object
dtype: object
Order_status: ['Success']
Ad_id: 0
```

In [13]:
```
# CELL 12
# Cek overlap ad_id antara kedua dataset
ads_id = set(marketing_ads_data['ad_id'])
transaction_id = set(sales_transaction_clean['ad_id'])

print("ad_id di marketing tapi gaada di transaction:", len(ads_id - transaction_id))
print("ad_id di transaction tapi gaada di marketing:", len(transaction_id - ads_id))
print("ad_id yang match:", len(ads_id & transaction_id))
```

```
ad_id di marketing tapi gaada di transaction: 413
ad_id di transaction tapi gaada di marketing: 1
ad_id yang match: 587
```

In [14]:
```
# CELL 13
# Drop duplikat kolom setelah merge dataset

merged_data = marketing_ads_data.merge(
    sales_transaction_clean,
    on='ad_id',
    how='left',
    suffixes=('', '_transaction')
)

merged_data = merged_data.drop(columns=['product_type_transaction', 'product_name_t
print("Shape:", merged_data.shape)
print("Columns:", merged_data.columns.tolist())

# Final Check pada merged_data sebelum analisis
print("\n === Final Check pada merged_data sebelum analisis ===")
print(merged_data.columns.tolist())
print(merged_data.head())
```

```
Shape: (2274, 16)
Columns: ['ad_id', 'campaign_name', 'audience_type', 'product_type', 'product_name',
'variant', 'date', 'impressions', 'clicks', 'spend_idr', 'sessions', 'add_to_cart',
'transaction_id', 'transaction_date', 'revenue_idr', 'order_status']

 === Final Check pada merged_data sebelum analisis ===
['ad_id', 'campaign_name', 'audience_type', 'product_type', 'product_name', 'varian
t', 'date', 'impressions', 'clicks', 'spend_idr', 'sessions', 'add_to_cart', 'transa
ction_id', 'transaction_date', 'revenue_idr', 'order_status']
     ad_id         campaign_name audience_type product_type  \
0  AD_0032    Awareness Campaign          cold        Serum
1  AD_0110    Awareness Campaign          cold    Sunscreen
2  AD_0137  Retargeting Campaign          warm    Sunscreen
3  AD_0137  Retargeting Campaign          warm    Sunscreen
4  AD_0137  Retargeting Campaign          warm    Sunscreen

                   product_name         variant       date  impressions  \
0    DermaGlow Brightening Serum    Video_Review 2023-01-06         8060
1  DermaGlow UV Shield Sunscreen    Video_Review 2023-09-01        17314
2  DermaGlow UV Shield Sunscreen  Static_Catalog 2023-04-13        16374
3  DermaGlow UV Shield Sunscreen  Static_Catalog 2023-04-13        16374
4  DermaGlow UV Shield Sunscreen  Static_Catalog 2023-04-13        16374

   clicks  spend_idr  sessions  add_to_cart transaction_id transaction_date  \
0     253  458896.00       216           29            NaN              NaT
1     542  108070.00       452           72            NaN              NaT
2     180  135435.00       146            8     TXN_001630       2023-04-14
3     180  135435.00       146            8     TXN_001628       2023-04-13
4     180  135435.00       146            8     TXN_001629       2023-04-16

   revenue_idr order_status
0          NaN          NaN
1          NaN          NaN
2    231000.00      Success
3    279000.00      Success
4    427000.00      Success
```

In [15]:

```python
# CELL 14
# Hitung metrik per ad_id
merged_data['CTR'] = merged_data['clicks'] / merged_data['impressions']
merged_data['CPC'] = merged_data['spend_idr'] / merged_data['clicks']

# Untuk conversion rate & ROAS, dilakukam aggregate per ad_id
# karena 1 ad_id bisa punya multiple transaksi

ad_summary = merged_data.groupby(['ad_id', 'variant', 'campaign_name', 'audience_ty
    impressions=('impressions', 'first'),
    clicks=('clicks', 'first'),
    spend_idr=('spend_idr', 'first'),
    sessions=('sessions', 'first'),
    add_to_cart=('add_to_cart', 'first'),
    total_revenue=('revenue_idr', 'sum'),
    total_transactions=('transaction_id', 'count')
).reset_index()

# Hitung metrik
```

```
ad_summary['CTR'] = ad_summary['clicks'] / ad_summary['impressions']
ad_summary['CPC'] = ad_summary['spend_idr'] / ad_summary['clicks']
ad_summary['ROAS'] = ad_summary['total_revenue'] / ad_summary['spend_idr']
ad_summary['conversion_rate'] = ad_summary['total_transactions'] / ad_summary['clic

print("Shape ad_summary:", ad_summary.shape)
print(ad_summary.head())
```

```
Shape ad_summary: (1000, 16)
     ad_id         variant        campaign_name audience_type product_type  \
0  AD_0001    Video_Review  Flash Sale Campaign          cold     Cleanser
1  AD_0002  Static_Catalog   Awareness Campaign          cold     Cleanser
2  AD_0003  Static_Catalog   Awareness Campaign          cold  Moisturizer
3  AD_0004  Static_Catalog  Flash Sale Campaign          cold        Serum
4  AD_0005    Video_Review  Flash Sale Campaign          cold  Moisturizer

   impressions  clicks  spend_idr  sessions  add_to_cart  total_revenue  \
0        18964     501  237337.00       408           23     1450000.00
1         7769     105  427069.00        86            6      723000.00
2        18096     189  256730.00       158            9           0.00
3         6899      83  131551.00        70           12      806000.00
4        15792     505  305041.00       423           22           0.00

   total_transactions  CTR     CPC  ROAS  conversion_rate
0                   5 0.03  473.73  6.11             0.01
1                   2 0.01 4067.32  1.69             0.02
2                   0 0.01 1358.36  0.00             0.00
3                   3 0.01 1584.95  6.13             0.04
4                   0 0.03  604.04  0.00             0.00
```

In [16]:

```
# CELL 15
# EDA 1 - Overall Performance: Video_Review vs Static_Catalog
overall = ad_summary.groupby('variant').agg(
    total_impressions=('impressions', 'sum'),
    total_clicks=('clicks', 'sum'),
    total_spend=('spend_idr', 'sum'),
    total_transactions=('total_transactions', 'sum'),
    total_revenue=('total_revenue', 'sum'),
    avg_CTR=('CTR', 'mean'),
    avg_CPC=('CPC', 'mean'),
    avg_ROAS=('ROAS', 'mean'),
    avg_conversion_rate=('conversion_rate', 'mean')
).reset_index()

print(overall)
```

```
          variant  total_impressions  total_clicks  total_spend  \
0  Static_Catalog            6077373         72240 160021363.50
1    Video_Review            6223089        155948 149334712.00

   total_transactions  total_revenue  avg_CTR  avg_CPC  avg_ROAS  \
0                1167   332312000.00     0.01  2538.63      3.40
1                 694   196570000.00     0.03  1117.39      1.77

   avg_conversion_rate
0                 0.02
1                 0.00
```

INSIGHT - EDA 1 Overall Performance

Static_Catalog outperform Video_Review in conversion & revenue, meskipun Video unggul di CTR. Ini menunjukkan Video efektif untuk awareness, sedangkan Static lebih kuat untuk conversion → gunakan funnel strategy (Video → Static).

In [17]:
```python
# CELL 16
# EDA 2 - Statistical  Significance Testing (T-test)

from scipy import stats
# Conversion: Transaksi berhasil vs tidak berhasil
static_catalog = ad_summary[ad_summary['variant'] == 'Static_Catalog']
video_review = ad_summary[ad_summary['variant'] == 'Video_Review']

# Total click vs total transactions per variant
static_converted = static_catalog['total_transactions'].sum()
static_not_converted = static_catalog['clicks'].sum() - static_converted

video_converted = video_review['total_transactions'].sum()
video_not_converted = video_review['clicks'].sum() - video_converted

# Contingency Table
contingency_table = [
    [static_converted, static_not_converted],
    [video_converted, video_not_converted]
]

chi2, p_value, dof, expected = stats.chi2_contingency(contingency_table)

print(f"Chi-Square: {chi2:.4f}")
print(f"P-value: {p_value:.4f}")
print(f"Degrees of freedom: {dof}")
print(f"\n Kesimpulan: {'Signifikan secara statistik (p < 0.05)' if p_value < 0.05
```

```
Chi-Square: 834.6519
P-value: 0.0000
Degrees of freedom: 1

 Kesimpulan: Signifikan secara statistik (p < 0.05)
```

INSIGHT EDA 2 - Statistical Significance

Perbedaan conversion rate antara Static_Catalog dan Video_Review signifikan (confidence >99.99%), menunjukkan Static_Catalog secara konsisten outperform dalam mendorong konversi. Prioritaskan Static_Catalog sebagai winning variant untuk scaling campaign karena keunggulannya terbukti secara statistik, bukan kebetulan.

In [18]:
```
# CELL 17
# EDA 3 - Segmentation per Product Type & Audience Type

# segmentation per Product Type
segment_product = ad_summary.groupby(['variant', 'product_type']).agg(
    total_transactions=('total_transactions', 'sum'),
    total_revenue=('total_revenue', 'sum'),
    avg_CTR=('CTR', 'mean'),
    avg_ROAS=('ROAS', 'mean'),
    avg_conversion_rate=('conversion_rate', 'mean')
).reset_index()

print("=== SEGMENTASI PER PRODUCT TYPE ===")
print(segment_product.to_string())

# segmentation per Audience Type
segment_audience = ad_summary.groupby(['variant', 'audience_type']).agg(
    total_transactions=('total_transactions', 'sum'),
    total_revenue=('total_revenue', 'sum'),
    avg_CTR=('CTR', 'mean'),
    avg_ROAS=('ROAS', 'mean'),
    avg_conversion_rate=('conversion_rate', 'mean')
).reset_index()

print("\n === SEGMENTASI PER AUCDIENCE TYPE ===")
print(segment_audience.to_string())
```

```
=== SEGMENTASI PER PRODUCT TYPE ===
          variant product_type  total_transactions  total_revenue  avg_CTR  avg_ROAS
avg_conversion_rate
0  Static_Catalog     Cleanser                 242    71890000.00     0.01      2.29
0.02
1  Static_Catalog  Moisturizer                 296    81721000.00     0.01      3.05
0.02
2  Static_Catalog        Serum                 289    80323000.00     0.01      3.70
0.02
3  Static_Catalog    Sunscreen                 340    98378000.00     0.01      4.35
0.02
4    Video_Review     Cleanser                 199    54203000.00     0.02      2.16
0.01
5    Video_Review  Moisturizer                 172    48013000.00     0.02      1.39
0.00
6    Video_Review        Serum                 161    48962000.00     0.03      2.04
0.00
7    Video_Review    Sunscreen                 162    45392000.00     0.03      1.60
0.00

 === SEGMENTASI PER AUCDIENCE TYPE ===
          variant audience_type  total_transactions  total_revenue  avg_CTR  avg_ROA
S  avg_conversion_rate
0  Static_Catalog          cold                 726   203494000.00     0.01      2.9
9                0.02
1  Static_Catalog          warm                 441   128818000.00     0.01      4.2
3                0.02
2    Video_Review          cold                 461   129654000.00     0.02      1.7
1                0.00
3    Video_Review          warm                 233    66916000.00     0.03      1.8
9                0.00
```

INSIGHT EDA 3 - Segmentation per Product type & Audience type

Static_Catalog unggul di semua segment, terutama Sunscreen (ROAS 4.35) dan warm audience (ROAS 4.23) → fokus scaling di segment high-intent ini.

INSIGHT EDA 4 - Trade Off antara CTR & Conversion

Terdapat trade-off jelas: Video_Review unggul di CTR, namun Static_Catalog menghasilkan conversion & revenue lebih tinggi. CTR tinggi pada Video menunjukkan interest awal, tapi belum tentu purchase intent. Static_Catalog lebih langsung ke produk & harga, sehingga klik yang masuk lebih berkualitas, jangan optimasi campaign hanya berdasarkan CTR, prioritaskan conversion & revenue, dan gunakan kombinasi Video (awareness) - Static (conversion).

A/B Testing Analysis Result

Key Takeaways

- Static_Catalog secara konsisten outperform Video_Review dalam conversion & revenue (ROAS 3.40 vs 1.77), dan perbedaannya terbukti signifikan secara statistik ($p < 0.001$).
- Video_Review unggul di CTR & CPC, namun lebih berfungsi sebagai awareness driver, bukan conversion driver.

Strategic Recommendation

- **Scale Static_Catalog sebagai main driver** Alokasikan ~70–80% budget ke Static_Catalog untuk memaksimalkan conversion & ROAS.
- **Double down pada Sunscreen + Static_Catalog** Segment ini memiliki performa tertinggi (ROAS 4.35) → jadikan sebagai hero campaign.
- **Prioritaskan warm audience untuk conversion** Static_Catalog ke warm audience menghasilkan ROAS tertinggi (4.23) → fokuskan retargeting di segment ini.
- **Gunakan Video_Review untuk top funnel** Alokasikan ~20–30% budget untuk awareness (cold audience), lalu arahkan ke retargeting dengan Static_Catalog.
- **Investigasi kategori Cleanser** Gap performa tipis, indikasi faktor lain (pricing, competition, atau demand) yang perlu dianalisis lebih lanjut.

In [19]:
```
# CELL 19
# EXPORTING CLEANED DATA

marketing_ads_data.to_csv('marketing_ads_data_clean.csv', index=False)
sales_transaction_clean.to_csv('sales_transaction_clean.csv', index=False)
merged_data.to_csv('merged_data.csv', index=False)

print("Export selesai!")
```

```
Export selesai!
```

In [20]:
```
# CELL 20
# Export ad_summary untuk visualisasi (filter outlier ROAS)

ad_summary_viz = ad_summary[ad_summary['spend_idr'] >= 50000].copy()

print("Shape:", ad_summary_viz.shape)
print(ad_summary_viz[['spend_idr', 'CTR', 'ROAS', 'conversion_rate']].head())

ad_summary_viz.to_csv('ad_summary_clean.csv', index=False)
print("Export selesai!")
```

```
Shape: (990, 16)
   spend_idr  CTR  ROAS  conversion_rate
0  237337.00 0.03  6.11             0.01
1  427069.00 0.01  1.69             0.02
2  256730.00 0.01  0.00             0.00
3  131551.00 0.01  6.13             0.04
4  305041.00 0.03  0.00             0.00
Export selesai!
```